

# Completion Caching: Reusing LLM Answers at Provider Scale

Adam Catto

## 1 Introduction

Large language model (LLM) serving is expensive for a reason that is easy to understate: most of the work is not “running the model on the question,” but *writing the answer*. A typical chat request has two phases. *Prefill* consumes the prompt in one (or a few) highly parallel forward passes of the serving Transformer: every prompt token is looked up in the model’s embedding table and then transformed by every layer, on the accelerator. *Decode* then emits tokens autoregressively; each step streams the same model weights—and, as the context grows, the key–value (KV) cache—from high-bandwidth memory. On contemporary accelerators this second phase is memory-bandwidth bound and has poor model-FLOP utilization (Leviathan et al., 2023; Pope et al., 2023). Section 2 will show that a short chat prefill is often memory-bound as well: the difference is that prefill streams the weights *once*, while decode streams them once *per output token*. For the short-to-medium prompts and medium-to-long answers that dominate consumer chat, decode is the term that occupies the chip, sets time-to-last-token, and, once a fleet is provisioned for peak tokens-per-second, sets the bill.

That would be merely an unfortunate fact about Transformers if every request asked something new. In practice it does not. Search engines learned decades ago that query streams are heavy-tailed: a modest number of intents account for a large fraction of volume, each intent appearing as many surface forms. Public chat corpora show the same shape. Topic mass concentrates (Zhao et al., 2024; Zheng et al., 2024); clusters of near-paraphrase are common; and for a surprising fraction of those clusters the *acceptable answers* form a basin, not a point. “How do I reverse a list in Python?”, “python reverse list in place”, and “what’s the idiomatic way to flip a list?” do not require three independently decoded completions. They require one good completion, reused.

This paper asks whether a provider that already sees this traffic—Google, or any other lab operating a frontier chatbot at scale—can stop paying decode for the head of that distribution. We call the resulting policy *completion caching*. The object in the cache is not a KV block or a prompt prefix. It is a pair  $(Q, C(Q))$ : a previously seen query and the completion the model already produced for it. Given a new query  $Q$ , the system must decide whether some cached completion  $C^*$  is an *acceptable* answer to  $Q$ . If so, it returns  $C^*$  and never decodes. A first decision can be made from a small retrieval embedder  $e(Q)$ , a nearest-neighbor lookup, and a gate—cheap enough to run on the host. If that gate is unsure, a serving-model prefill of  $Q$  supplies a hidden state  $h(Q)$  in the model’s own geometry and a second gate can still skip decode. If that gate rejects, the model decodes as usual and the new pair may be inserted.

### 1.1 What is already cached, and what is not

Serving stacks already cache aggressively. None of the standard caches is aimed at this decision.

Prefix and paged KV caches (Kwon et al., 2023) reuse attention state when two requests share a token prefix. Prompt-cache products do the same at the API boundary. These techniques are valuable: they cut prefill, improve time-to-first-token, and make multi-turn and templated workloads cheaper. They do not skip decode. After the shared prefix is restored, the model still writes a new answer, one bandwidth-bound step at a time. Prefill–decode disaggregation (Zhong et al., 2024) makes that fact architectural—the two phases even live on different machines—but it still *pays* for decode.

Exact-match response caches (hash the prompt, return the bytes) are the other extreme. They skip both phases, but only when the user types the same string again, and they are unsafe even then: two users can type “best Italian restaurant near me” and must not receive each other’s answer. Paraphrase without a shared context is the typical portable case; paraphrase *with* a bound context is a different object, treated in Section 3.2.

Semantic caches such as GPTCache (Bang, 2023), and more recent context-aware variants (Yan et al., 2025; Yu et al., 2025), are the closest prior work. They embed the query, retrieve a stored response by vector similarity, and, on a hit, bypass the model entirely. That is the right systems instinct, and production deployments have shown large latency wins *when the hit is correct*. The limitation is the decision rule. Cosine similarity between query embeddings is a proxy for “these questions are about the same thing.” It is not a proxy for “this stored completion is an acceptable answer to *this* question.” Near-neighbors in embedding space include near-misses that matter: different versions (“install Python 3.11” versus “3.12”), different entities with similar form (“capital of France” versus “capital of Spain”), and questions that share a topic but not an answer. Application-layer caches also typically treat the LLM as a black box. They do not use the model’s own prefill state, they rarely degrade into a cheaper decode when the match is only partial, and they are seldom accompanied by a hardware cost model that says when a lookup is actually cheaper than generating.

A third line of work reuses *text* without claiming the full answer is reusable. Retrieval-based speculative decoding (He et al., 2024) and prompt-lookup decoding draft tokens from a datastore or from the prompt itself; the model still verifies every token. These methods convert redundancy into a speedup with no quality change. They are complementary: a cached completion that is not safe to serve wholesale can still be a strong draft.

Completion caching sits between these lines. Like a semantic cache, it tries to skip decode. Like a serving-stack cache, it is willing to spend a prefill—a signal the model is already designed to compute—before making that call. Like speculative retrieval, it can fall back to using  $C^*$  as a draft rather than as an answer. The criterion it optimizes is not query similarity and not prefix equality. It is *answer acceptability*.

## 1.2 The idea

Write  $C(\cdot)$  for the serving model’s completion map, and write  $\mathcal{A}(Q, C)$  for the (unobserved) predicate that  $C$  is an acceptable reply to  $Q$ . A completion cache is a set of pairs  $\{(Q_i, C_i)\}$  with  $C_i = C(Q_i)$  at insertion time. On a new query  $Q$  the system must either return some  $C_i$  for which  $\mathcal{A}(Q, C_i)$  holds, or report a miss and decode  $C(Q)$ .

$\mathcal{A}$  is not computable from  $Q$  alone, and treating  $\mathcal{A}(Q, C_i)$  as “ $Q$  is close to  $Q_i$  in embedding space” is exactly the error mode above. The useful observation is that we have richer cheap signals than that:

- a retrieval embedding  $e(Q)$  from a small dual encoder (tens to hundreds of millions of parameters), enough to retrieve a shortlist and, when the match is overwhelming, to return a cached completion without touching the serving model;

- the serving model’s last-token prefill state  $h(Q)$ , already computed if decode is about to start, and a query representation in the model’s own geometry;
- lightweight gates on  $(e(Q), e(Q_i), h(Q), h(Q_i), C_i)$  that can be trained or calibrated to be conservative: a false hit serves the wrong answer, and is much more expensive than a false miss, which only costs a decode that would have been run anyway.

Those signals suggest a three-tier policy, specified in Section 3. A high-confidence, *portable* hit never touches an accelerator. An ambiguous shortlist is resolved by prefilling  $Q$  and gating on  $h(Q)$ ; a pass still skips decode. A reject falls through to ordinary generation, optionally using the retrieved completion as a speculative draft. Context-bound intents (location, account, live search) are not served from another user’s  $C_i$ ; they may still skip decode by replaying a cached tool plan into a completion template.

Two further mechanisms are worth naming here because they change the hit-rate calculus. *Top-K Projective Pooling* treats the cache not as a single nearest neighbor but as a small set of completions that can be selected among, voted on, or (in a more speculative design) fused. Many intents have several acceptable answers; requiring a unique nearest neighbor leaves hits on the table. *Gating* is the opposite pressure: it exists to keep the served set inside  $\mathcal{A}$ . The systems problem here is transformed from maximizing cosine recall to maximizing decode-skips subject to a bound on unacceptable serves. *Top-K Projective Pooling* is a technique for solving this problem; *Gating* is a policy for choosing among the solutions.

### 1.3 Why this is worth doing at provider scale

Three facts have to be true at once for completion caching to matter. We argue that for a large consumer chatbot they are.

First, decode really is the scarce resource. Section 2 makes this quantitative across GPU and TPU parts, but the qualitative picture is stable. Prefill of a hundred-token question is one parallel pass. Decode of a three-hundred-token answer is hundreds of sequential, bandwidth-bound steps. Batching amortizes weight traffic across requests and is the main reason serving is viable; it does not make skipped decode free, because a skipped request is a slot that another request can occupy. At fleet scale, hit rate times decode cost is accelerator-hours.

Second, the lookup is cheap relative to that cost. Embedding a short query and searching an HNSW or IVF index of  $10^7$ – $10^9$  vectors is a CPU-and-DRAM problem, typically milliseconds, and is straightforward to colocate with the request router. Returning a few hundred tokens of cached text is a network problem measured in the same units. The comparison that matters is not “is retrieval free?” It is “is retrieval plus a conservative gate cheaper than occupying an HBM-bound chip for the length of the answer?” For any plausible chat length, yes—provided the hit is real.

Third, the traffic is skewed enough that a conservative gate still sees volume. We do not need, and should not claim, that “most” queries are duplicates. We need the head of the intent distribution—how-to questions, definitions, boilerplate code, exam-style problems, recipes, consumer product questions—to be large in absolute terms. A provider serving hundreds of thousands of queries per second can justify a substantial cache engineering effort at a single-digit-to-low-double-digit hit rate, if those hits are concentrated on decode-heavy, low-personalization requests. The long tail still decodes. That is expected; it is also how every successful cache works.

The same facts explain where the idea is *not* worthwhile. Long-context workloads whose cost is prefill (summarize this document; answer from this unique corpus) gain little from a completion cache: the query is unique and decode may be the smaller term. Deeply personalized or tool-using turns (“what’s on my calendar”; “refactor *this* file”) are not acceptable to serve from someone

else’s  $C_i$ . Time-sensitive questions go stale. A model-version bump invalidates the cache, or at least requires a freshness policy. These are design constraints, not objections to the head of the distribution.

## 1.4 What this paper does

This is a design paper. We do not yet report a production deployment. The contribution is to treat completion reuse as a first-class serving decision and to work out whether, and under what policy, it pays.

Section 2 develops a napkin cost model for prefill and decode on GPU and TPU, including the host and network terms a cache introduces. Section 3 specifies the method: the serve path, gating, top- $k$  projective pooling, and the miss-side draft and insert rules. Section ?? translates hit rate and gate conservatism into fleet-level savings.

The claim we are willing to defend is modest and, we think, correct. At provider scale, a non-trivial fraction of chat decode is spent writing answers that already exist in the fleet’s own history. Deciding that—carefully, mostly on CPU, and with prefill as a refinement rather than as a commitment to generate—is one of the few optimizations that can remove decode rather than merely accelerate it.

## 2 Prefill, decode, and where the FLOPs actually go

Completion caching decides whether to return a stored answer before, during, or after the serving model’s forward pass. The cost of each option depends on what that pass actually does. This section accounts for the work on a public dense stand-in, Llama 3.1 70B (Grattafiori et al., 2024), whose width, depth, and grouped-query layout are published. A frontier Gemini-class model is wider and deeper; the same identities apply, with different  $P$ ,  $L$ , and  $d$ .

### 2.1 Computations on the path to an answer

A request  $Q$  can incur three computations before any completion token is written.

**Tokenizer.** A byte-pair or SentencePiece tokenizer maps the UTF-8 string  $Q$  to integer ids  $t_1, \dots, t_S \in \{1, \dots, V\}$ . This is a host-side finite-state walk: no Transformer weights, well under a millisecond on a short question, and irrelevant to the cost model.

**Retrieval embedder  $e(Q)$ .** A small dual encoder—MiniLM-class, on the order of  $2 \times 10^7$  parameters, or a larger e5/GTE-class model of a few times  $10^8$ —produces a single vector for nearest-neighbor lookup in the completion cache. It is a Transformer, just not the serving model. Its forward pass is two to three orders of magnitude cheaper than a 70B prefill (Table 1), so it can live on the host next to the request router. A provider that prefers to run it on a small accelerator does not change that comparison.

**Serving-model prefill.** The serving model reads the same token ids, gathers rows of its embedding table  $E \in \mathbb{R}^{V \times d}$ , and pushes the resulting  $S \times d$  residual stream through all  $L$  decoder blocks (Pope et al., 2023). Every linear layer participates. The pass writes a KV cache into HBM and, if generation is about to start, may apply the language-model head  $W_{\text{lm}} \in \mathbb{R}^{d \times V}$  to the last position to obtain first-token logits. A cache gate that reads the last-token hidden state  $h(Q) \in \mathbb{R}^d$  does not need  $W_{\text{lm}}$ .

Table 1: Work on a short chat question. The retrieval embedder  $e(Q)$  is a separate small model. Prefill and decode use the serving Transformer.

| Computation               | Work                                        | Device   | Order of FLOPs ( $S=128$ ) |
|---------------------------|---------------------------------------------|----------|----------------------------|
| Tokenizer                 | string $\rightarrow$ ids                    | host CPU | none (table lookup)        |
| Retrieval embedder $e(Q)$ | small Transformer, $P_e \sim 2 \times 10^7$ | host CPU | $\sim 6 \times 10^9$       |
| Serving prefill $h(Q)$    | Llama 3.1 70B, all $L$ layers               | GPU/TPU  | $\sim 1.8 \times 10^{13}$  |
| One decode step           | same 70B weights, one new token             | GPU/TPU  | $\sim 1.4 \times 10^{11}$  |

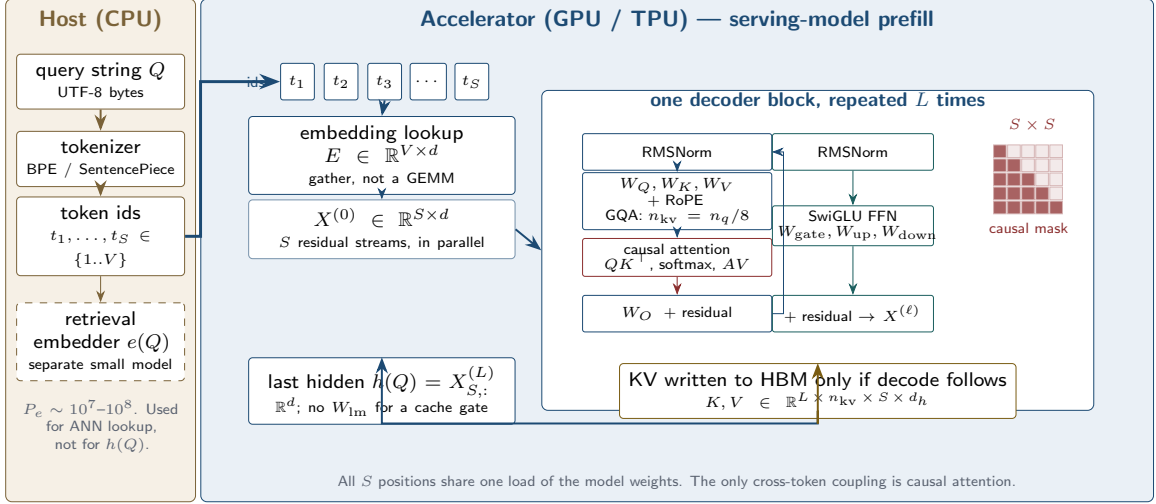


Figure 1: Anatomy of serving-model prefill. The host only tokenizes (and, optionally, runs a small retrieval embedder  $e(Q)$ ). On the accelerator, token ids become an  $S \times d$  residual stream by embedding lookup. Each of the  $L$  decoder blocks applies RMSNorm, grouped-query projections, rotary positions, a causal  $S \times S$  attention, an output projection, and a SwiGLU feed-forward, with residuals around both sub-layers. All  $S$  positions are transformed in one weight load. The cache gate reads the last row  $h(Q)$ ; the language-model head is unnecessary for that decision. The KV tensors are written to HBM only if decode will follow.

The last two rows set the economics. Prefill of a 128-token question does about as much *arithmetic* as 126 decode steps, because it applies the body weights to 128 positions at once. It does not take 126 times as long. Both phases are dominated by streaming the same  $\sim 140$  GB of BF16 weights off HBM; prefill does it once, decode does it once per emitted token.

## 2.2 What the serving model does in prefill

Figure 1 is that pass for a Llama-3-style decoder (Ainslie et al., 2023; Grattafiori et al., 2024).

Let  $d$  be the residual width,  $n_q$  the number of query heads,  $n_{\text{kv}}$  the number of key/value heads,  $d_h = d/n_q$  the head dimension, and  $d_{\text{ff}}$  the SwiGLU inner width. For Llama 3.1 70B these are

$$L = 80, \quad d = 8192, \quad n_q = 64, \quad n_{\text{kv}} = 8, \quad d_h = 128, \quad d_{\text{ff}} = 28672, \quad V = 128256.$$

One block, at every position  $s = 1, \dots, S$  and in parallel across  $s$ , computes

$$\begin{aligned}
\tilde{x}_s &= \text{RMSNorm}(x_s), \\
Q_s &= \tilde{x}_s W_Q, \quad K_s = \tilde{x}_s W_K, \quad V_s = \tilde{x}_s W_V, \\
Q_s, K_s &\leftarrow \text{RoPE}(Q_s, K_s), \\
a_s &= \text{softmax}\left(\frac{Q_s K_{\leq s}^\top}{\sqrt{d_h}}\right) V_{\leq s}, \\
x_s &\leftarrow x_s + a_s W_O, \\
x_s &\leftarrow x_s + \text{SwiGLU}(\text{RMSNorm}(x_s); W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}).
\end{aligned} \tag{1}$$

Grouped-query attention (Ainslie et al., 2023) is the only structural peculiarity:  $W_K$  and  $W_V$  map to  $n_{\text{kv}} d_h = 1024$  dimensions rather than  $d$ , and each KV head is shared by  $n_q/n_{\text{kv}} = 8$  query heads. That shrinks the KV cache by  $8\times$  relative to multi-head attention.

Two implementation facts matter for caching. First, the serving model’s embedding table  $E$  is a gather on the accelerator,  $t_s \mapsto E_{t_s}$ , and is then dominated by the  $L$  blocks that follow; it is not a substitute for the retrieval embedder  $e(Q)$ . Second, because every position is materialized, prefill emits  $h(Q) = X_{S,:}^{(L)}$  with no extra pass. If the gate accepts a cached completion, the newly written KV cache is discarded and decode never starts. If the gate rejects, that KV cache is exactly what decode would have built, so the prefill is not wasted.

### 2.3 Concrete arithmetic on Llama 3.1 70B

A matrix multiply  $Y = XW$  with  $X \in \mathbb{R}^{S \times d_{\text{in}}}$  and  $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$  costs  $2Sd_{\text{in}}d_{\text{out}}$  FLOPs (multiply-add). Summing the seven linear maps in (1) over  $L$  layers gives the body parameter count and the linear FLOPs:

$$\begin{aligned}
P_{\text{attn}} &= d(n_q d_h + 2n_{\text{kv}} d_h + d) = 1.510 \times 10^8, \\
P_{\text{ffn}} &= 3d d_{\text{ff}} = 7.046 \times 10^8, \\
P_{\text{body}} &= L(P_{\text{attn}} + P_{\text{ffn}}) = 6.845 \times 10^{10}, \\
F_{\text{lin}}(S) &= 2P_{\text{body}} S.
\end{aligned} \tag{2}$$

The embedding table and the untied lm-head add  $2Vd = 2.101 \times 10^9$  parameters, bringing the public 70.6B total (Grattafiori et al., 2024). Causal attention adds a quadratic term that is still small at chat lengths:

$$F_{\text{attn}}(S) = 4Ln_q d_h S^2 = 2.621 \times 10^6 S^2. \tag{3}$$

RMSNorm, SiLU, softmax, and RoPE are  $O(LSd)$  and do not move any table below. For a cache gate we omit  $W_{\text{lm}}$ , so

$$F_{\text{prefill}}(S) = F_{\text{lin}}(S) + F_{\text{attn}}(S) = 1.369 \times 10^{11} S + 2.621 \times 10^6 S^2. \tag{4}$$

One decode step at context length  $c$ , *including* the lm-head on the new token, is

$$F_{\text{dec}}(c) = 2P_{\text{body}} + 4Ln_q d_h c + 2dV \approx 1.390 \times 10^{11} + 2.621 \times 10^6 c. \tag{5}$$

Table 2 is the FLOP picture. A 128-token prefill is 17.6 TFLOP. A 256-token completion is about 36 TFLOP. Counting FLOPs alone, decode is only twice prefill, and one might conclude that skipping decode is a  $2\times$  win. That conclusion is wrong, because the two phases do not have the same arithmetic intensity.

Table 2: Worked FLOP counts for Llama 3.1 70B. Attention is negligible on short chat prompts and only a few percent even at  $S = 4096$ . Decode of  $T$  tokens costs about  $T$  times a single-token body pass, plus a cheap walk over the growing KV cache.

| Quantity                                    | $S=128$               | $S=512$               | $S=4096$              |
|---------------------------------------------|-----------------------|-----------------------|-----------------------|
| Linear FLOPs $F_{\text{lin}}$               | $1.75 \times 10^{13}$ | $7.01 \times 10^{13}$ | $5.61 \times 10^{14}$ |
| Attention FLOPs $F_{\text{attn}}$           | $4.29 \times 10^{10}$ | $6.87 \times 10^{11}$ | $4.40 \times 10^{13}$ |
| Prefill total (no lm-head)                  | $1.76 \times 10^{13}$ | $7.08 \times 10^{13}$ | $6.05 \times 10^{14}$ |
| Attention share of prefill                  | 0.24%                 | 0.97%                 | 7.3%                  |
| One decode step at $c=S$                    | $1.39 \times 10^{11}$ | $1.40 \times 10^{11}$ | $1.50 \times 10^{11}$ |
| $T=256$ decode steps, $c$ from $S$ to $S+T$ | $3.57 \times 10^{13}$ | $3.59 \times 10^{13}$ | $3.83 \times 10^{13}$ |

Table 3: Peak arithmetic, HBM bandwidth, and ridge point. H100 BF16 is the *dense* tensor-core rate; NVIDIA’s advertised 1979 TFLOP/s includes 2:4 sparsity, which ordinary dense checkpoints do not use. TPU numbers are per chip (Austin et al., 2025).

| Part            | Peak (BF16) | Peak (FP8/INT8) | HBM BW    | $I^*$ (BF16) |
|-----------------|-------------|-----------------|-----------|--------------|
| NVIDIA H100 SXM | 989 TFLOP/s | 1979 TFLOP/s    | 3.35 TB/s | 295 FLOP/B   |
| TPU v5e         | 197 TFLOP/s | 394 TFLOP/s     | 0.82 TB/s | 240 FLOP/B   |
| TPU v5p         | 459 TFLOP/s | 918 TFLOP/s     | 2.8 TB/s  | 164 FLOP/B   |
| TPU v6e         | 920 TFLOP/s | 1840 TFLOP/s    | 1.6 TB/s  | 575 FLOP/B   |

**Bytes, not FLOPs, set the clock.** In BF16 the body plus embed plus head occupy

$$B_w = 2P \approx 1.41 \times 10^{11} \text{ bytes}$$

(141 GB). Prefill reads those weights once. Decode reads them once *per output token*. The KV cache is a rounding error at chat length:  $K$  and  $V$  together occupy  $4Ln_{\text{kv}}d_hS = 3.28 \times 10^5 S$  bytes, or 40 MB at  $S = 128$  and 1.3 GB at  $S = 4096$ . Activation traffic for the residual stream is  $2Sd$  bytes per layer crossing, also small against 141 GB.

The roofline intensity against weight traffic, for batch  $B$  and a multiply-add of  $2P$  FLOPs per token, is therefore just the number of tokens that share a weight load (Pope et al., 2023; Williams et al., 2009):

$$I_{\text{prefill}} = BS, \quad I_{\text{dec}} = B \quad (\text{FLOP/byte, BF16}). \quad (6)$$

A chip is compute-bound when  $I$  exceeds its ridge point  $I^* = \text{peak FLOP/s/HBM bytes/s}$ , and memory-bound otherwise.

On an H100,  $I^* \approx 295$  in dense BF16 (Table 3). Equation (6) then says:

- Decode is memory-bound until the decode batch approaches  $B \approx 300$ . A 70B KV cache will not let a single chip get there.
- Prefill of a short question is also memory-bound at batch one:  $S = 128 < 295$ . It becomes compute-bound once  $BS \gtrsim 295$ —for example  $B = 3$  at  $S = 128$ , or  $B = 1$  at  $S = 512$ .

The slogan “prefill is compute-bound, decode is memory-bound” is true of long prompts and of batched prefills. It is not true of a lone 64-token question. What remains true, and what the cache exploits, is the sequentiality. Prefill pays the weight-stream *once*. Decode pays it  $T$  times.

Table 4: Roofline lower bounds for one request, Llama 3.1 70B,  $S = 128$ ,  $T = 256$ . “Chip-ms” at batch  $B$  is wall time of the batched kernel divided by  $B$ : the accelerator-time charged to the request. Prefill at  $B = 32$  is compute-bound on H100 FP8 ( $BS = 4096 > I^*$ ); decode is not.

|                            | H100, batch 1             |       | H100 FP8, batch 32 |             |
|----------------------------|---------------------------|-------|--------------------|-------------|
|                            | BF16                      | FP8   | per batch          | per request |
| Prefill, $S=128$           | 42 ms                     | 21 ms | 285 ms             | 8.9 ms      |
| Decode, $T=256$            | 10.8 s                    | 5.4 s | 5.4 s              | 168 ms      |
| Decode share of chip-time  | 99.6%                     | 99.6% | 95%                | 95%         |
| CPU $e(Q)$ + ANN (typical) | 1–10 ms, not on this chip |       |                    |             |

**Time lower bounds.** A memory-bound kernel cannot beat  $B_w/\text{BW}$ . A compute-bound kernel cannot beat  $F/\text{peak}$ . Taking the max is the standard bound (Pope et al., 2023). For Llama 3.1 70B, batch one, BF16, on one H100,

$$\begin{aligned}
 t_{\text{mem}} &= 141 \text{ GB} / 3.35 \text{ TB/s} = 42 \text{ ms}, \\
 t_{\text{comp}}^{\text{prefill}}(S=128) &= 17.6 \text{ TFLOP} / 989 \text{ TFLOP/s} = 18 \text{ ms}, \\
 t_{\text{comp}}^{\text{dec}} &= 0.14 \text{ TFLOP} / 989 \text{ TFLOP/s} = 0.14 \text{ ms}.
 \end{aligned}$$

So a 128-token prefill and a single decode step have the *same* 42 ms floor. A 256-token completion has a 10.8 s floor. FP8 halves the weight footprint to 71 GB and halves those floors to 21 ms and 5.4 s; the ratio  $T:1$  does not change. Tensor-parallel shards of the same model change the per-chip bytes and add all-reduce, but they do not change the fact that decode repeats the stream.

TPU v5p tells the same story with different constants. One chip has 96 GB of HBM, so a BF16 70B must be sharded across at least two chips; an FP8 70B fits on one. Per-chip bandwidth is 2.8 TB/s, slightly below H100, and  $I^*$  is lower (164), so short prefills are if anything *more* memory-bound. TPU v5e (16 GB, 0.82 TB/s) is a prefill-friendly small chip and would never hold this model alone; v6e raises peak FLOPs faster than bandwidth ( $I^* = 575$ ), which makes short prefills even more firmly a weight-stream. On every one of these parts, decode is the term a cache can delete.

A 700 W H100 burning 42 ms for prefill spends about 30 J. The same chip burning 5.4 s for an FP8 256-token decode spends about 3.8 kJ. Energy follows chip-time.

**The retrieval embedder, for scale.** MiniLM-L6 has  $P_e \approx 2.3 \times 10^7$  parameters (Wang et al., 2020). Its forward cost  $2P_e S \approx 5.8 \times 10^9$  FLOPs at  $S = 128$  is  $3000\times$  smaller than the 70B prefill. A 335M embedder is still  $\sim 200\times$  smaller. Either one plus an HNSW probe of  $10^7$ – $10^9$  vectors is a few milliseconds on the host (Malkov & Yashunin, 2020). Returning a 256-token cached string is a few kilobytes; inside a datacenter that is sub-millisecond. These terms matter for latency tails. They do not appear in the accelerator bill.

## 2.4 Implications for completion caching

Table 4 says that at chat lengths, decode is 95–99.6% of serving-model accelerator time. A hit that prefills  $Q$ , inspects  $h(Q)$ , and returns  $C^*$  therefore discards almost all of the request’s chip-time. A more precise gate that still skips decode is cheap: the extra prefill costs on the order of one decode token. The host-side embedder  $e(Q)$  is cheaper still, and is the only way to leave the serving accelerators untouched. That last property matters when the fleet is prefill-saturated, or when

prefill and decode are disaggregated onto different machines (Zhong et al., 2024). In a disaggregated stack a prefill-then-accept hit still occupies a prefill chip and still frees a decode chip. Decode chips scale with  $T$  and are hard to keep at high utilization; freeing them is the point.

Those ratios give the three tiers of Section 3 a cost:

1. **Retrieval tier.** Compute  $e(Q)$  on the host. If a cached  $C_i$  passes a conservative gate on  $(e(Q), e(Q_i))$ , return  $C_i$ . Serving-accelerator time: zero.
2. **Prefill tier.** Otherwise prefill  $Q$ , obtain  $h(Q)$ , and gate on the serving model’s own state. On accept, discard the KV cache and return  $C_i$ . Accelerator time: one weight-stream.
3. **Decode tier.** On reject, continue from the KV cache just written and generate  $C(Q)$ , optionally drafting from  $C_i$ . Accelerator time: the usual  $T$  streams. Insert  $(Q, C(Q))$  if the request was cache-eligible.

Long-context and unique-document requests stay in the decode tier, and often should not be inserted: their cost is prefill, their  $Q$  will not recur, and  $h(Q)$  represents a document the next user does not have. Context-bound intents (location, account, live search) are not portable completions; Section 3.5 reuses a tool plan rather than another user’s  $C_i$ . The napkin math is for portable decode-skips and for bound plans that avoid decode—short  $Q$ , longish  $C$ , repeated intents—where  $T$  weight-streams are the thing worth not paying.

### 3 Completion caching

Section 2 priced the three places a request can leave the serving path: after a host-side retrieval, after a serving-model prefill, or after a full decode. This section specifies the policy that chooses among them. The object we optimize is not query similarity. It is decode-skips subject to a bound on serving an unacceptable answer. An answer can be unacceptable because it is about the wrong thing, or because it is about the right thing *for someone else*.

#### 3.1 Cache records and the serve path

A completion cache is a set of records

$$\mathcal{D} = \{ r_i = (Q_i, C_i, e_i, h_i, [Q_i], \kappa_i, \pi_i, \mathcal{T}_i, m_i) \}.$$

$C_i = C(Q_i)$  at insertion;  $e_i = e(Q_i)$  is the retrieval embedding;  $h_i = h(Q_i)$  is the serving model’s last-token prefill state when it was stored (optional).  $[Q_i]$  is a query equivalence-class representative (Section 3.2).  $\kappa_i$  is a binding class: portable, bound, or ineligible. For bound records,  $\pi_i$  is a tool plan (a program with holes) and  $\mathcal{T}_i$  is a completion template with the same holes. Metadata  $m_i$  holds insertion time, model version, TTL, and an optional quality score.

Write  $\mathcal{A}(Q, C)$  for the (unobserved) predicate that  $C$  is an acceptable reply to  $Q$ . Acceptability is not a property of strings alone. It requires compatible *intent* and compatible *context*: the stored answer must be one this user, at this time and place, could have been given. Algorithm 1 is the serve path. Figure 2 is the same policy as a control flow.

Exact string match is therefore not a return path. It is at most a candidate, and only after  $n(\cdot)$  has folded trivial surface variation and BindClass has declared the intent portable. Retrieval still returns a shortlist of  $k > 1$  records. The gate remains ternary on portable traffic: ACCEPT is calibrated to keep the served set inside  $\mathcal{A}$ ; REJECT is the decode we would have run anyway; DEFER spends a prefill, which Section 2 priced at about one decode token.

---

**Algorithm 1** Serve a query against a completion cache.

---

**Require:** query  $Q$ , user/session context  $u$ , cache  $\mathcal{D}$ , embedder  $e$ , serving model, integers  $k, r$

```
1:  $[Q] \leftarrow n(Q)$  ▷ equivalence-class representative
2:  $\kappa \leftarrow \text{BindClass}(Q, u)$  ▷ portable | bound | ineligible; cheap, possibly learned
3: if  $\kappa = \text{ineligible}$  then
4:   return  $\text{Generate}(Q)$  ▷ do not insert
5: end if
6: if  $\kappa = \text{bound}$  then
7:   return  $\text{FillBound}(Q, u, [Q], \mathcal{D})$  ▷ tool plan + template; Section 3.5
8: end if
9: if  $\exists r \in \mathcal{D}$  with  $[Q_r] = [Q]$ , fresh, same model,  $\kappa_r = \text{portable}$  then
10:  if  $\text{CheapGate}(Q, u, r) = \text{ACCEPT}$  then
11:    return  $C_r$  ▷ still gated: identical text is not a license
12:  end if
13: end if
14:  $e_Q \leftarrow e(Q)$ ;  $\mathcal{N} \leftarrow \text{kNN}(e_Q, \mathcal{D}; k)$ 
15:  $(C^*, d) \leftarrow \text{PoolAndGate}(e_Q, \mathcal{N}, u; \text{space } e)$ 
16: if  $d = \text{ACCEPT}$  then
17:   return  $C^*$  ▷ retrieval tier
18: end if
19: if  $d = \text{REJECT}$  then
20:   return  $\text{MaybeInsert}(Q, \text{Generate}(Q), u)$ 
21: end if
22:  $(h_Q, \text{KV}) \leftarrow \text{Prefill}(Q)$  ▷  $d = \text{DEFER}$ 
23:  $(C^*, d) \leftarrow \text{PoolAndGate}(h_Q, \mathcal{N}, u; \text{space } h)$ 
24: if  $d = \text{ACCEPT}$  then
25:   discard KV; return  $C^*$ 
26: end if
27: return  $\text{MaybeInsert}(Q, \text{Decode}(\text{KV}; \text{draft } C^*), u)$ 
```

---

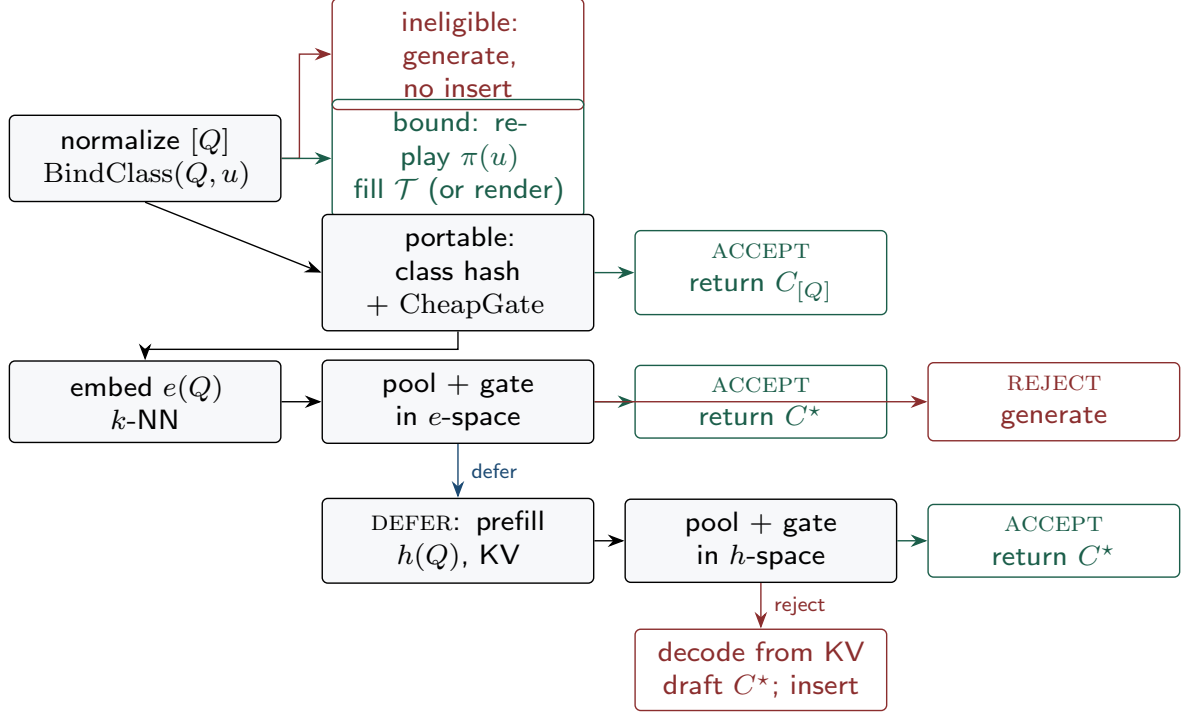
### 3.2 Equivalence classes and context binding

Two strings can be the same question, and the same question can demand two different answers.

**Equivalence.** Write  $n(Q)$  for a representative of the query’s equivalence class  $[Q]$ . The map  $n$  is a stack, cheapest first:

1. Unicode NFKC, case folding, strip trailing punctuation, collapse whitespace.
2. A small, closed rewrite list: contractions (*what’s*  $\mapsto$  *what is*), a handful of locative paraphrases (*near me* / *around me* / *nearby* / *close by*).
3. Optional learned equivalence: a classifier or a very conservative embedding threshold trained on labeled paraphrase pairs, used only to merge  $[Q]$ , never to authorize serving  $C_i$ .

Thus “What is the best Italian restaurant near me”, the same sentence with a question mark, “what’s the best Italian restaurant near me”, and “what are the best Italian restaurants around me” can share a class. That is useful for lookup. It is not a decision to reuse a completion. The restaurant class is a single intent whose *answer* depends on who asked.



Green leaves skip serving-model decode. Bound fill-in may still call tools. Exact class-hash still goes through CheapGate.

Figure 2: Serve path. Binding class is decided before any completion is reused. Portable queries may return a cached  $C^*$  after a cheap gate, including on an equivalence-class hash. Bound queries replay a tool plan into a template under *this* user’s context. Ineligible queries decode and are not stored. DEFER buys a serving-model prefill to decide again in  $h$ -space.

**Binding.**  $\text{BindClass}(Q, u)$  assigns  $\kappa \in \{\text{portable}, \text{bound}, \text{ineligible}\}$ . Portable intents have answers that do not depend on the requester (how to reverse a list in Python; what RMSNorm is). Bound intents have a stable *shape* and a per-request *payload*: location, account, calendar, the open document, a live index. Ineligible intents have neither a reusable answer nor a reusable shape (a unique pasted file with no tool schema; a jailbreak; a request the provider has chosen not to cache).

The classifier can be a lexicon and a few regular expressions to start—“near me”, “my”, “I have”, “today”, “this file”, session identifiers in the prompt—and should be allowed to become a small learned model. The cost of running it is in the same budget as  $e(Q)$ : host-side, milliseconds. The cost of getting it wrong in the portable direction is serving one user another user’s restaurant, medical detail, or file. So  $\text{BindClass}$  is conservative toward **bound** and **ineligible**. A false **bound** still skips decode if a tool plan exists; a false **portable** is the failure we are unwilling to buy.

A stored record carries its own  $\kappa_i$ . A portable completion is reusable only for a portable query in the same  $[Q]$  (or a pooling accept, Section 3.4). A bound record does not export  $C_i$ . It exports  $\pi_i$  and  $\mathcal{T}_i$ .

### 3.3 Gating

A false accept serves the wrong answer. A false reject only costs a decode. Gates are therefore conservative by construction: we bound  $\Pr(\neg \mathcal{A} \mid \text{ACCEPT})$  and let hit rate fall where it must.

CheapGate on an equivalence-class hash is the same doctrine at lower feature cost: binding class, freshness, model version, and a small context check against  $u$ . Identical tokens do not skip that gate.

Let  $x_Q$  be the query representation in the current space ( $e_Q$  at the retrieval tier,  $h_Q$  at the prefill tier) and let  $\mathcal{N} = \{r_i\}_{i=1}^k$  be the shortlist. The gate  $g$  reads  $x_Q$ , the matching fields of  $\mathcal{N}$ , and  $u$ , and returns  $(\hat{C}, d)$  with  $d \in \{\text{ACCEPT}, \text{DEFER}, \text{REJECT}\}$ . We do not treat  $\cos(x_Q, x_i)$  as  $g$ . Cosine on raw embeddings is a proxy for “these questions are about the same thing,” and that is the France/Spain failure mode from Section 1.1. The features we do use are:

- **Binding compatibility.** If  $\kappa$  or  $\kappa_i$  is not **portable**,  $g$  cannot accept  $C_i$ . Bound shortlists are handed to Section 3.5.
- **Projected residual**  $\rho$  and projected neighbor  $i^*$  from top- $k$  projective pooling (Section 3.4). A small residual means  $Q$  lies in the local paraphrase span of  $\mathcal{N}$ . A large residual means the shortlist is a topical near-miss.
- **Completion consensus**  $\gamma$ : do the  $k$  stored answers agree with each other? A mixed shortlist is a collision of intents that happen to sit nearby in representation space.
- **Freshness and version.** A record with an expired TTL, or a model version other than the one now serving, cannot be accepted as  $C_i$ . It may still donate a draft or a template.

A simple, deployable  $g$  on portable traffic is a threshold rule on  $(\rho, \gamma)$  after those hard filters:

$$d = \begin{cases} \text{ACCEPT} & \text{if } \rho \leq \tau_\rho^{\text{hi}}, \gamma \geq \tau_\gamma, \kappa = \kappa_i = \text{portable}, \\ \text{REJECT} & \text{if } \rho > \tau_\rho^{\text{lo}} \text{ or a hard filter fails,} \\ \text{DEFER} & \text{otherwise, and only at the retrieval tier.} \end{cases} \quad (7)$$

At the prefill tier there is no further deferral: DEFER collapses to REJECT and decode starts from the KV cache just written. The two thresholds  $\tau_\rho^{\text{hi}} < \tau_\rho^{\text{lo}}$  leave a band in which  $e$ -space is unsure and  $h$ -space is worth buying. Both  $\tau$  and  $\tau_\gamma$  are calibrated offline on labeled (paraphrase, near-miss, bound) triples so that the empirical false-accept rate stays under a chosen  $\varepsilon$  (for example  $10^{-3}$ ). A learned classifier on the same features is a drop-in replacement for (7); the calibration target does not change.

The served completion on ACCEPT is not necessarily the nearest neighbor. It is the output of the pooling rule below.

### 3.4 Top- $k$ projective pooling

Nearest-neighbor caches treat the shortlist as a ranked list and take the head. That wastes two facts. Many portable intents have several acceptable answers, so a unique neighbor leaves hits on the table. And paraphrase variation is approximately low-rank in a local neighborhood, while the coordinates that change the *answer*—a version number, an entity, a negation—are often the ones that stick out of that neighborhood. Top- $k$  projective pooling uses the shortlist as a local basis rather than as a leaderboard. It is a rule for portable records. Bound records are pooled in plan-and-template space, not in  $C_i$  space (Section 3.5).

**Local paraphrase subspace.** Let  $x_1, \dots, x_k$  be the shortlist representations in the current space and  $\bar{x} = \frac{1}{k} \sum_i x_i$ . Form the centered matrix  $X = [x_1 - \bar{x} \mid \dots \mid x_k - \bar{x}]$  and take the top  $r$  left singular vectors  $U_r$  (typically  $r = \min(3, k - 1)$ ). Write  $P = U_r U_r^\top$  for the projector onto that span. The query residual is

$$\rho(Q, \mathcal{N}) = \frac{\|(I - P)(x_Q - \bar{x})\|}{\|x_Q - \bar{x}\| + \epsilon}. \quad (8)$$

A small  $\rho$  means  $Q$  is a displacement inside the neighborhood—the usual signature of paraphrase. A large  $\rho$  means  $Q$  has a component the neighbors do not share. That is the signature of a near-miss that cosine still ranks highly (“Python 3.11” versus “3.12” when both sit near a pile of “install Python” questions).

The projected neighbor is

$$i^* = \arg \min_{i \leq k} \|P(x_Q - \bar{x}) - P(x_i - \bar{x})\|. \quad (9)$$

Distances in the span compare phrasing; the residual in (8) is what decides whether that comparison is about the same question.

**Consensus.** Let  $\phi(C_i)$  be a cheap embedding of the stored completions (the same  $e$ , or even a bag-of-tokens hash). Cluster  $\{\phi(C_i)\}$  with a single-linkage cutoff. Write  $\gamma$  for the fraction of the shortlist that lands in the largest cluster, and write  $\mathcal{N}_{\text{maj}}$  for that cluster. If  $\gamma$  is small, the neighborhood is mixed: the local span includes more than one answer, and accepting  $C_{i^*}$  is accepting a coin flip between them. Those shortlists never accept at the retrieval tier. They may still accept at the prefill tier, where  $h(Q)$  can break the tie, or they fall through to decode.

**The pooled completion.** On ACCEPT,

$$C^* = C_{j^*}, \quad j^* = \arg \min_{r_j \in \mathcal{N}_{\text{maj}}} \|P(x_Q - \bar{x}) - P(x_j - \bar{x})\|, \quad (10)$$

breaking remaining ties by freshness and stored quality. This is selection, not generation: we do not fuse portable text. Fusion is attractive on paper and unsafe in a conservative cache. The “pooling” is of evidence—the shortlist votes on whether an acceptable answer is present, and on which stored answer to serve.

Two failure modes are worth naming because they set  $k$  and  $r$ . If  $k$  is too small, the estimated span  $U_r$  is noisy and  $\rho$  is meaningless. If  $k$  is too large, the shortlist reaches into neighboring intents,  $\gamma$  drops, and the gate defers or rejects. If  $r$  is too large,  $P$  becomes a near-identity and  $\rho$  cannot see a near-miss. The useful regime is a small neighborhood that is still large enough to estimate a two- or three-dimensional paraphrase plane:  $k \in [8, 32]$ ,  $r \in \{2, 3\}$  is the intended operating point. Those constants are for the designer to sweep; they are not claims about a deployed system.

### 3.5 Bound queries: tool plans, templates, and non-AR fill-in

A bound intent is a program the serving model has already written, plus a sentence the model has already found for presenting the result. Caching  $C_i$  copies another user’s result into this user’s thread. Caching the program does not.

**Tool plans.** When a bound request is generated the first time, the serving model typically emits one or more tool calls—maps search, catalog lookup, calendar, code execution—then a natural-language wrap. MaybeInsert stores, in addition to or instead of  $C_i$ , a plan  $\pi$ : a sequence of tool names and argument *templates* whose holes are bound at serve time from  $u$  (user coordinates, account id, “now”, the active document). Replaying  $\pi(u)$  is ordinary RPC. It is not decode. The restaurant class becomes `places.search(cuisine=italian, near = u.geo)`, not “try Mario’s on Oak Street.”

**Templates.** The wrap around a tool result is low-entropy given the slots. From a shortlist of bound records in  $[Q]$  we extract a completion template  $\mathcal{T}$  with holes aligned to  $\pi$ ’s outputs: *The highest-rated Italian place near you is {name} ({rating}) at {addr}*. Extraction can be heuristic (longest common subsequence with slots masked) or learned. FillBound then does

$$C = \mathcal{T}(\pi(u)), \quad (11)$$

string interpolation, not generation. If several templates in the class disagree, take the majority skeleton or fall through to decode. Consensus here is consensus of *shapes*, not of filled-in names.

**Non-autoregressive fill-in.** Interpolation is brittle when the wrap is not quite a template—a slightly different tone, a hedge, an extra clause. A small conditional generator that maps  $(\mathcal{T}, \pi(u))$  to a surface string, trained offline on logged (template, slots, completion) triples, can sit in that gap. Discrete or latent diffusion models for text are one candidate: they are not token-serial in the serving-model sense, and they can be run on a cheap device next to the router. We treat this as an optional renderer, not as a second chatbot. It does not see  $u$  except through the already-fetched slots; it is not authorized to invent a restaurant. If a confidence check on the renderer fails, the request falls through to serving-model decode.

FillBound is therefore a fourth leave-point, cheaper than prefill when the plan hits and still conservative: the only tokens we reuse are those that do not mention a person, and the only facts we insert were fetched for this request.

### 3.6 Decode, draft, and insert

On a portable miss, the serving model generates  $C(Q)$  as usual. Two things are still left to decide: whether the retrieved completion can cheapen that generate, and what to write back to  $\mathcal{D}$ .

**Speculative draft.** If PoolAndGate produced a  $C^*$  that it would not accept as an answer—a high cosine with a large residual, or a mixed shortlist— $C^*$  is still a plausible draft. Retrieval-based speculative decoding (He et al., 2024; Leviathan et al., 2023) verifies a draft in one forward pass per chunk and keeps a prefix that matches the model’s own distribution. Using a cached completion as that draft costs no extra model and cannot change the output. It converts a rejected cache hit into a partial decode-skip. The method therefore degrades smoothly: wholesale reuse when  $\mathcal{A}$  is confident, token-level reuse when it is not.

**Insertion.** MaybeInsert is binding-class-specific. Portable, short-to-medium, non-document  $Q$  store  $(Q, C, e, h, [Q], \text{portable}, m)$ . Bound  $Q$  store  $[Q], \kappa = \text{bound}$ , the tool plan  $\pi$ , and the template  $\mathcal{T}$ ; they do not store a filled  $C$  as a reusable answer. Ineligible  $Q$  store nothing. Time-sensitive portable facts (sports scores, prices) get a TTL or they are reclassified as bound and given a refresh tool. Model version is part of the key for  $h_i$  and  $C_i$ ;  $e_i, [Q], \pi$ , and  $\mathcal{T}$  can be rebuilt or kept across

a bump. Eviction is ordinary: LRU or LFU on the retrieval index, with a floor for records that have already paid for themselves in decode-skips.

The insert rule is as important as the gate. A cache that absorbs other people’s locations and unique documents will have a high occupancy and a low, dangerous hit rate. The cost model in Section 2 is for portable decode and for bound plans that avoid decode. That is the population we store.

## References

- Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 4895–4901. <https://doi.org/10.18653/v1/2023.emnlp-main.298>
- Austin, J., Douglas, S., Frostig, R., Levskaya, A., Chen, C., Vikram, S., Lebron, F., Choy, P., Ramasesh, V., Webson, A., & Pope, R. (2025). How to scale your model [See “How to Think About TPUs”]. <https://jax-ml.github.io/scaling-book/>
- Bang, F. (2023). GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, 212–218. <https://doi.org/10.18653/v1/2023.nlposs-1.24>
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., et al. (2024). The llama 3 herd of models. <https://doi.org/10.48550/arXiv.2407.21783>
- He, Z., Zhong, Z., Cai, T., Lee, J. D., & He, D. (2024). REST: Retrieval-based speculative decoding. *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 1582–1595. <https://doi.org/10.18653/v1/2024.naacl-long.88>
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626. <https://doi.org/10.1145/3600006.3613165>
- Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. *Proceedings of the 40th International Conference on Machine Learning, 202*, 19274–19286. <https://proceedings.mlr.press/v202/leviathan23a.html>
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., & Dean, J. (2023). Efficiently scaling transformer inference. *Proceedings of the Sixth Conference on Machine Learning and Systems*, 606–624. [https://proceedings.mlsys.org/paper\\_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html](https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html)
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, 33, 5776–5788. <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>

- Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76. <https://doi.org/10.1145/1498765.1498785>
- Yan, J., Ni, W., Chen, L., Lin, X., Cheng, P., Qin, Z., & Ren, K. (2025). ContextCache: Context-aware semantic cache for multi-turn queries in large language models. *Proceedings of the VLDB Endowment*, 18(12), 5391–5394. <https://doi.org/10.14778/3750601.3750679>
- Yu, C., Wang, T., Shao, Z., & Jiang, S. (2025). SmartCache: Context-aware semantic cache for efficient multi-turn LLM inference. *Advances in Neural Information Processing Systems*, 38, 191160–191181. <https://doi.org/10.52202/085713-5740>
- Zhao, W., Ren, X., Hessel, J., Cardie, C., Choi, Y., & Deng, Y. (2024). WildChat: 1m ChatGPT interaction logs in the wild. *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=Bl8u7ZRlbM>
- Zheng, L., Chiang, W.-L., Sheng, Y., Li, T., Zhuang, S., Wu, Z., Zhuang, Y., Li, Z., Lin, Z., Xing, E. P., Gonzalez, J. E., Stoica, I., & Zhang, H. (2024). LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=BOfDKxfwt0>
- Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., & Zhang, H. (2024). DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation*, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>